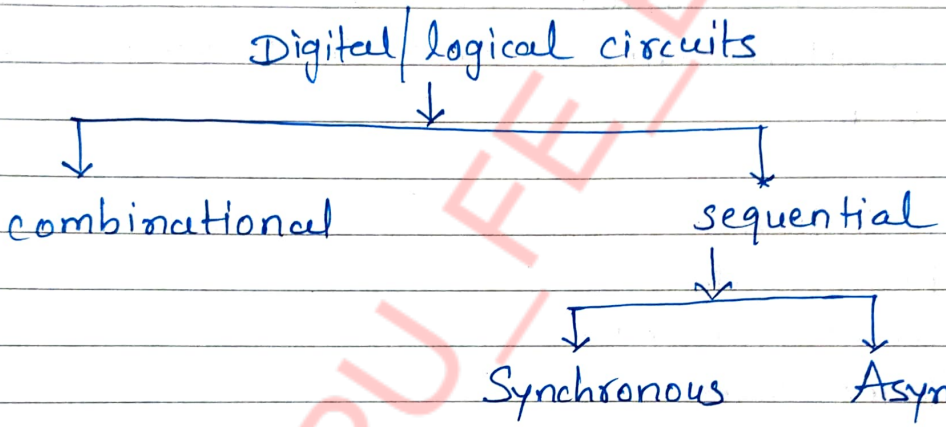
UNIT-II Combinational Logic Design.

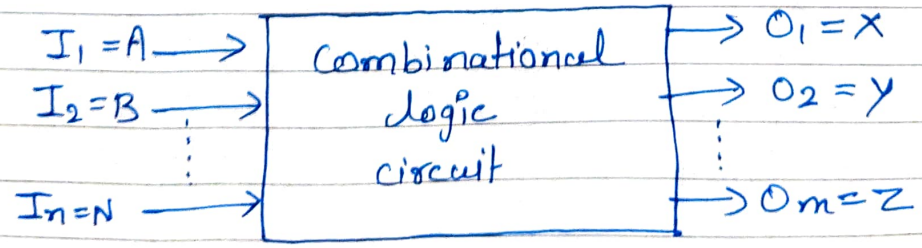
☆ Logic Circuits:-

- Digital circuits or logical circuits are classified into two types: combinational circuits and sequential circuits.
- Sequential circuits are further divided into two groups: synchronous and Asynchronous circuits.
- The Following diag. shows the classification of logic circuits.



☆ Combinational Circuit :-

- Combinational circuits uses logic gates to implement or satisfy a given Boolean Expression.
- The outputs of combinational circuit is a logical function of the input variables.
- This can be shown as in Fig.



TG:SPPU_FE_ENGG

PDF BY VL







✨ WHY JOIN ?

🏆 Trusted by 80,000+ Engineering Students
🔔 Stay Updated with latest university news
🚀 Make your engineering journey easy with us

Best Regards, Vishwajeet Londhe

© 2026 Vishwajeet Londhe. All Rights Reserved.

Study material provided by: V. B. L

Join Community by clicking below links

Telegram Channel

https://t.me/SPPU_FE_ENGG

(for all engineering Resources)

SPPU Engineering & Technical UPDATES

WhatsApp channel

WhatsApp Channel

(for all Engg & tech updates)

https://whatsapp.com/channel/0029ValjFriICVfpcV9HFc3b

Insta Page

(for all Engg & tech updates)

@SPPU_ENGINEERING_UPDATE

https://www.instagram.com/sppu_engineering_update

SPPU FE - ENGG

- The combinational logic circuit has only logic gates and does not require Memory. It's operation depends upon the present states of inputs and not on the previous states of the inputs.

⇒ Codes and Code converters:-

Binary code:-

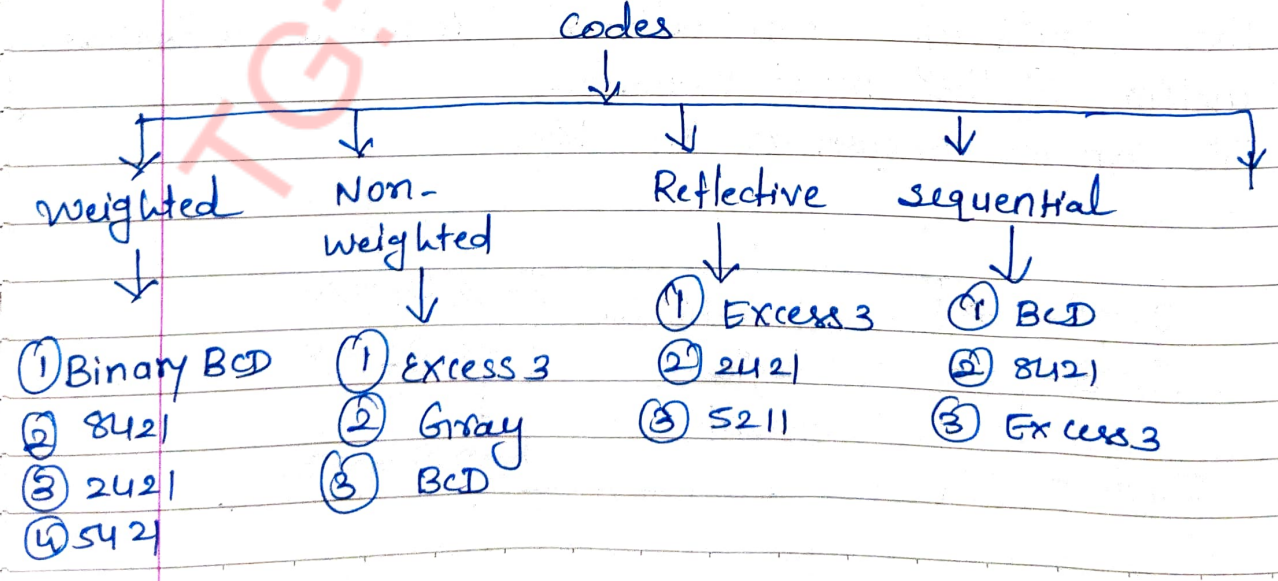
The binary code is represented by the number as well as alphanumeric letter.

It is suitable for digital communication.

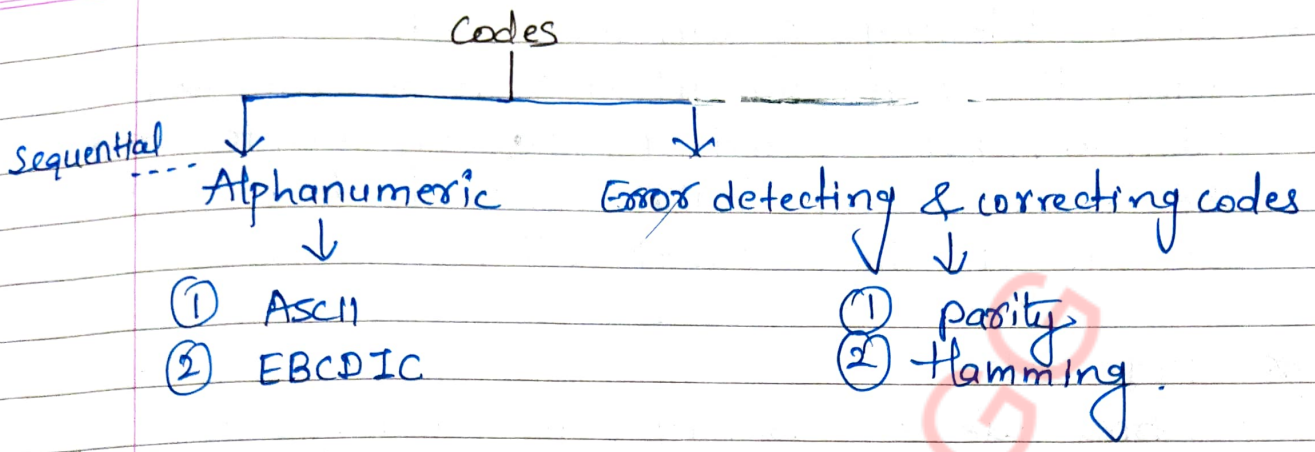
Implementation is easy since only 0 & 1 are used.

Classification:-

1) Weighted codes
2) Non-weighted codes
3) Reflective codes
4) Sequential codes
5) Alphanumeric codes
6) Error detecting codes
7) Error correcting codes



TG: SPPU_FE_ENGG



1) weighted codes → It obey the positional weight principle.

e.g. = 3 =

positional weight → 8 4 2 1

Code → 0 0 1 1

2) Non-weighted codes - positional weights are not assigned.

3) Reflective code - If the code for any number is the complement of any another number.

e.g. - code for '8' is 1000 where as the code for 1 is 0001, so both are complement of each other.

4) Sequential codes - codes in which 2 subsequent numbers in binary representation differ by only one digit.

5) Alphanumeric codes - codes that represent number & alphabetic characters.

26 letters & 10 digits i.e. total 36 items.

① American standard code for Information Interchange (ASCII)

② Extended Binary coded Decimal Interchange code (EBCDIC)

TE-SPPU-RE-ENGG

6) Error detecting & correcting codes – Used to detect the error present in the received data bitstream. Error correcting codes are used to correct the Error present in the received data, so that we can get the original data.

★ BCD Codes :~

BCD code is Binary coded decimal code.

In BCD we use the binary number from 0000 - 1001 i.e. from 0-9.

If the number contains two decimal digits then its equivalent BCD will be the respective Eight binary of the given decimal number.

i.e. four for the first digit & next four for the second decimal digit.

e.g. $(12)_{10}$ = 0001 0010

largest 4-bit code is 1001

The numbers 1010, 1011, 1100, ... 1101, 1110, 1111 are called forbidden numbers.

These 6 code combinations are not used in BCD.

★ BCD Addition :~

Rules :~

① Add two number using rules of Binary addition.

② If the 4-bit sum is less than 9 or equal to 9 then sum is in proper form.

③ If the 4-bit result is greater than 9 & if a carry bit is present in the result then it is invalid & we have to add '6' = $(0110)_2$ to the result of addition.

TG: SPPU_ENGG

Ex:- Add $(57)_{10}$ & $(26)_{10}$ in BCD

soln:- To Express $(57)_{10}$ & $(26)_{10}$ into BCD

$(57)_{10}$	0101	0111	
$(26)_{10}$	0010	0110	
	0111	1101	Invalid BCD NO.
	7	D	D>9

So, add $6 = (0110)_2$ into addition for 'D'

	0111	1101
+		0110
	1000	0011
	8	3

Valid BCD Result.

$\therefore (57)_{10} + (26)_{10} = (83)_{10}$ in BCD.

BCD Subtraction:-

BCD Subtraction by 9's complement Method -

1) First find the decimal Equivalent of the given BCD codes.
2) Carry 9's complement of the negative number.
3) Add this result to the number from which the subtraction is to be done
4) If there is carry on bit, then the carry on bit may be added to the result of the subtraction.

Ex:- Find (01010001) − (00100001)

soln:-

$(01010001)_2 = (51)_{10}$

$(00100001)_2 = (21)_{10}$

The 9's complement of neg. num. is.

$99 - 21 = 78$

Add complemented Value to 51,

$51 + 78 = 129$

TG-SPPU-FE-ENGG

In this result the MSB i.e. 1 is the carry on to 29,

$29 + 1 = 30$ which is the final answer of BCD

The Final result is,

$$(0101\ 0001)_{BCD} - (0010\ 0001)_{BCD} = (0011\ 0000)_{BCD}$$

Excess-3 code :~

1) These are non-weighted binary decimal codes.
2) self-complimentry codes.
3) Use Biased representation.
4) codes 0000 & 1111 are not used for any digits which is advantage for Memory organisation.

Representation of Excess-3 code :~

1) Find the decimal Equivalent of the given binary number.
2) Add +3 to each digit of decimal number.
3) Convert the newly obtained decimal number back to binary number to get required Excess-3 Equivalent.

Decimal No.	BCD	Excess-3
0	0 0 0 0	0 0 1 1
1	0 0 0 1	0 1 0 0
2	0 0 1 0	0 1 0 1
3	0 0 1 1	0 1 1 0
4	0 1 0 0	0 1 1 1
5	0 1 0 1	1 0 0 0
6	0 1 1 0	1 0 0 1
7	0 1 1 1	1 0 1 0
8	1 0 0 0	1 0 1 1
9	1 0 0 1	1 1 0 0

TG.SPRJ_EE_ENGG

M T W T F S S
Page No.
Date
YOUVA

Ex:- Convert decimal number 23 to Excess-3 code.

Soln:-

We add '3' to both digit in the decimal number

$23 + 33 = 56$

Convert into 4-bit binary number -

5 6

0101 0110

☆ Excess -3 Addition :-

1) First convert the numbers into Excess 3 form by adding 0011 with each of the four bit groups.
2) Now the two numbers are added using the basic laws of binary addition.
3) Now which of the four groups have produced a carry up, we have to add 0011 with them.
4) Subtract 0011 from the group which have not produced a carry during the addition.

Ex:- Add 9 & 4.

Soln:-

$9 \rightarrow 1001$

$4 \rightarrow 0100$

convert the numbers in Excess-3

$$\begin{array}{r} {\scriptstyle 1\ 1} \\ 1001 \\ +0011 \\ \hline 1100 \end{array} \qquad \begin{array}{r} 0100 \\ +0011 \\ \hline 0111 \end{array}$$

$\therefore (9)_{XS-3} = 1100$ $\quad \therefore (4)_{XS-3} = (0111)$

TG:SPPU_ENGG

Add the results,

```
   1                    We add as '3' as carry is '1'
  1 1 0 0                 1 1          1 1
+ 0 1 1 1               +   1      0 0 1 1
-------------           -------    -------
[1] 0 0 1 1             0 0 1 1    0 0 1 1
                        -------    -------
                        0 1 0 0    0 1 1 0
                                   (13)x5-3
                        Excess-3 for 13
```

$(13)_{x5-3} = (0100\ 0110)$

Ex:-2 Add 2 & 4.

Soln:- $2 \rightarrow 0010$

$4 \rightarrow 0100$

Convert into Excess-3

```
  1                    1 1 1
 0 0 1 0      0 1 0 0    0 1 0 1
+ 0 0 1 1    + 0 0 1 1  + 0 1 1 1
-----------  ----------  ----------
 0 1 0 1      0 1 1 1    1 1 0 0
```

Result of Addition, Here there is no carry on, so we subtract 3 or add 1101

```
   1
  1 1 0 0
  1 1 0 1
-----------
[1] 1 0 0 1      (1001) Excess3 for 6.
```

★ Gray code:-

It is sequence of binary number systems, which is also known as 'Reflected binary code'.

Here, the first $N/2$ values compared with those of that last $N/2$ values in reverse order, hence it is called as reflected binary code.

Two Consecutive value differ by one bit of binary digits.

TG:SPPU_FE_ENGG

It is used in general sequence of Hardware-generated binary numbers.

✡ Generation of gray codes~

1) We find the number of bits required to represent a number.
2) Then we find the code for '0' i.e. 0000, which is the same as binary.
3) Now, we consider previous code, i.e. 0000 & change the MSB of it.
4) We carry on this process till all the codes are not uniquely identified.
5) If by changing the MSB, we find the same code obtained previously, then the 2nd MSB will be changed, & so on.

⇒ Half Adder and Full Adder :~

- Addition is the most basic arithmatic operation in digital circuit.
- The simple addition of two bits has the following truth-table.

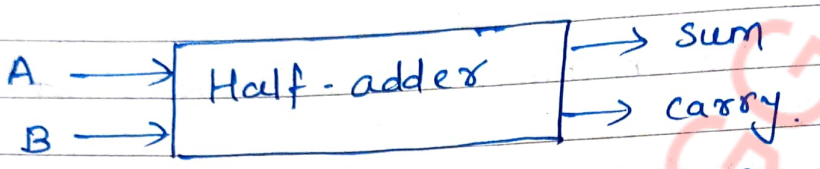
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

- Note that when both the inputs are 1's then 'sum' is '0' and one carry is generated, similar to decimal addition.

TG:SPUTTE_ENGG

HALF-ADDER

- The half-adder has two inputs (A, B) and two outputs (sum, carry)
- The block diagram of half-adder is shown in Fig.



- When the circuit has two or more outputs, one K-map is plotted for each output.
- The K-maps for half adder are:

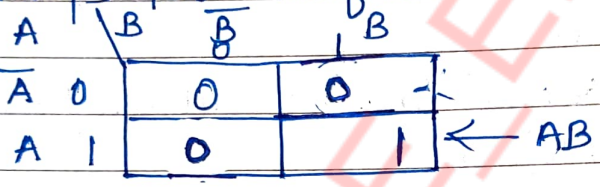


Fig. : K-Map for carry.

- Carry Output = AB

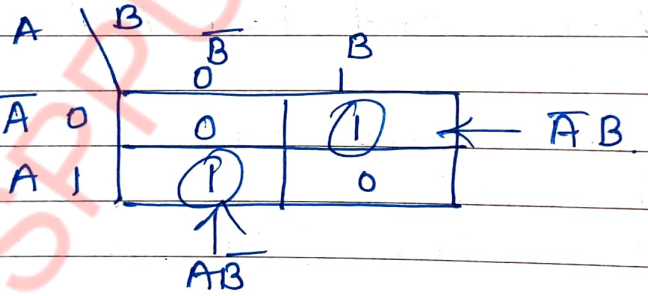


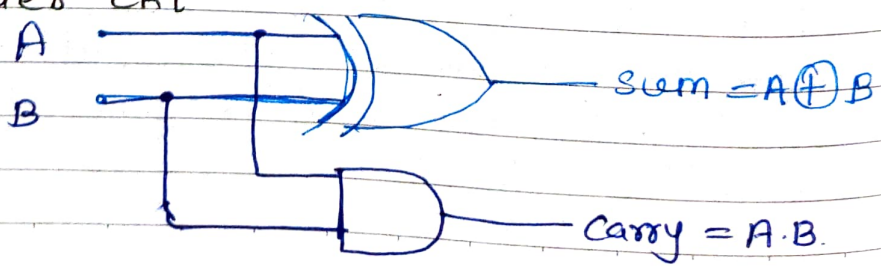
Fig:- K-map for sum o/p.

$\therefore$ Sum $= \bar{A}B + A\bar{B}$

$= A \oplus B$

↑ XOR

The Half adder ckt



TG: SPPU FE ENGG

FULL ADDER:-

- Full-Adder has an additional input of previous carry in.
- Full-Adder is a combinational circuit that forms the arithmetic sum of theree input bits i.e. A, B, Cin and produces two outputs, sum, Cout.
- The logic block diagram of full adder is shown in Fig.

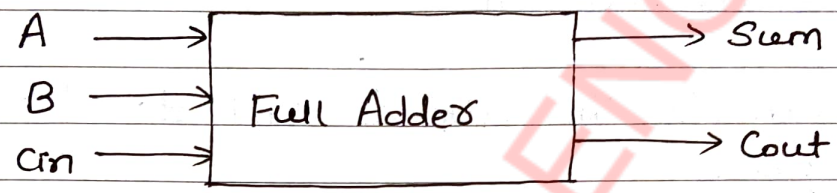
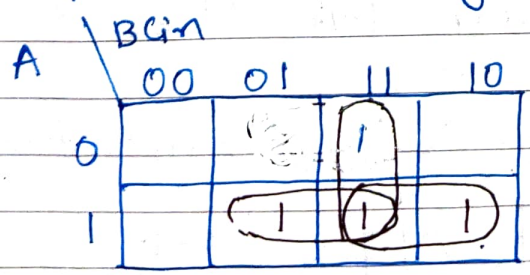


Fig. Block diagram of Full Adder

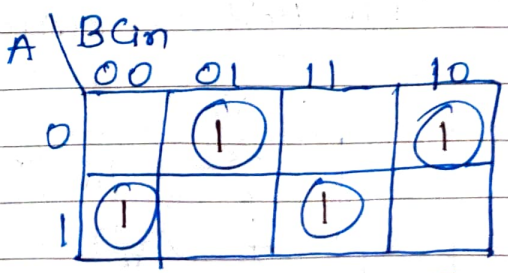
The truth table of full-adder

Inputs			outputs	
A	B	Cin	Cout	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

- Full-adder truth table is plotted on two k-maps corresponding to two outputs, Cout & sum



Cout = AB + BCin + ACin



Sum = A ⊕ B ⊕ Cin

TG: SPPU_FE_ENGG

$$Sum = A\bar{B}\bar{C}in + \bar{A}\bar{B}Cin + ABCin + \bar{A}B\bar{C}in$$

$$Sum = \bar{A}\bar{B}Cin + \bar{A}B\bar{C}in + A\bar{B}\bar{C}in + ABCin$$

$$= Cin(\bar{A}\bar{B} + AB) + \bar{C}in(\bar{A}B + A\bar{B})$$

$$= Cin(\overline{A \oplus B}) + \bar{C}in(A \oplus B)$$

$$= Cin(\bar{X}) + \bar{C}in\,X$$

$$= Cin \oplus X$$

$$Sum = A \oplus B \oplus Cin$$

$\bar{A}B + A\bar{B} = A \oplus B$, with X substituted for $A \oplus B$.

- The sum is the XORed output of A, B and Cin inputs.

Full Adder circuit:-

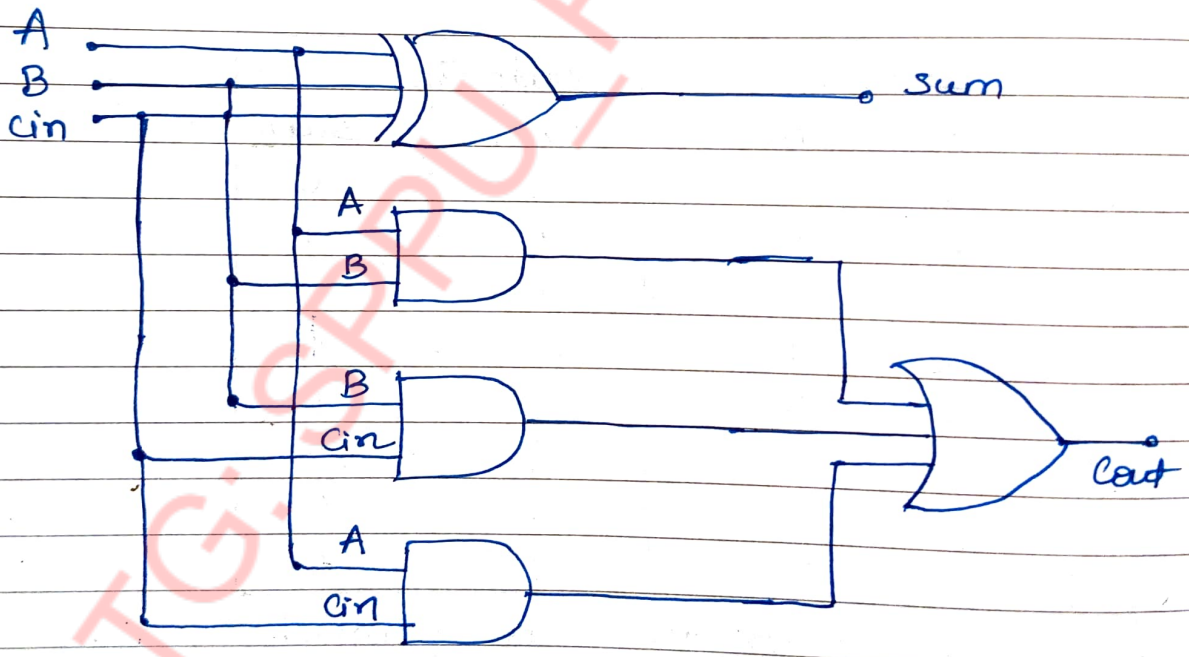
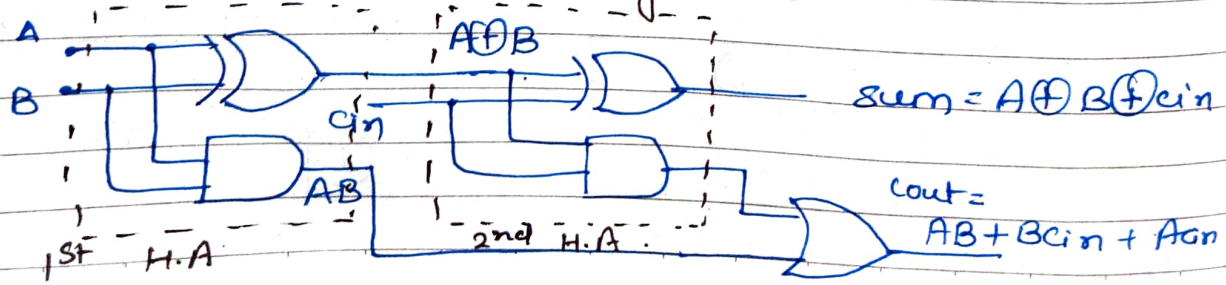


Fig. Full adder logic diagram.

⇒ Full-Adder circuit using Half-Adder.



TG:SPPU_FE_ENGG

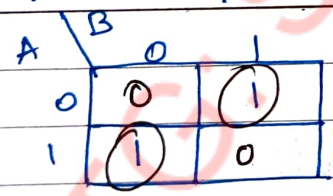
For k-Map Simplification:

$$C_{out} = AB + AC_{in} + BC_{in}$$
$$= AB + C_{in}(A+B)$$
$$= AB + C_{in}(A\bar{B} + \bar{A}B)$$
$$C_{out} = AB + C_{in}(A \oplus B)$$

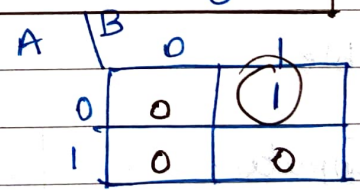
⇒ Half and Full subtractor:

- Subtractor :- The subtraction of two binary numbers is performed by combinational circuits called as half-subtractor and full-subtractor.
- Half-Subtractor :- Half-subtractor is a combinational logic circuit which subtracts two one-bit binary numbers. The truth-table and k-maps for corresponding outputs are as shown in Table.

Inputs		Outputs	
A	B	D	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0



(a) K-map for difference



(b) K-Map for Borrow

Fig. Truth-Table and K-maps of Half-subtractor

- From the K-Maps for the boolean expressions for the outputs are written as;

$$D = \bar{A}B + A\bar{B} = A \oplus B$$
$$B = \bar{A}B$$

TSSPU FREAKS

- Therefore, the logic circuit for half-subtractor is drawn as,

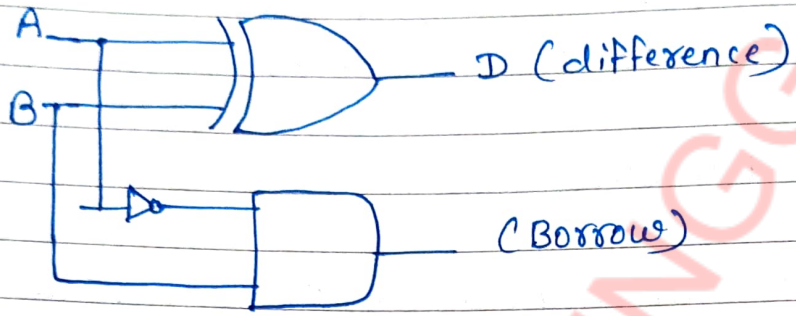
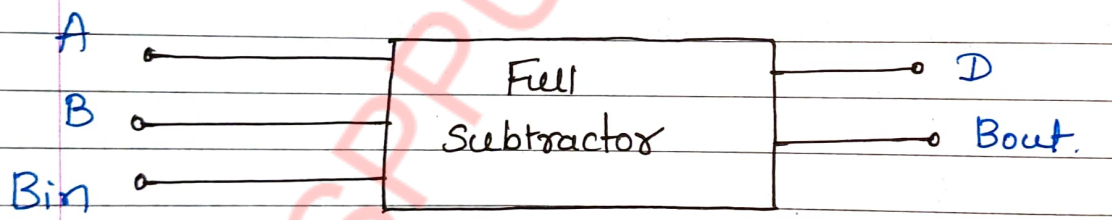


Fig. Half-subtractor.

✡ FULL SUBTRACTOR :-

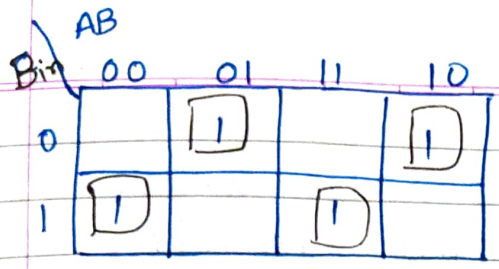
Full subtractor allows to subtract a borrow which may be generated form lower order bit subtraction. A simple block schematic of Full subtractor is shown in Fig. below:-



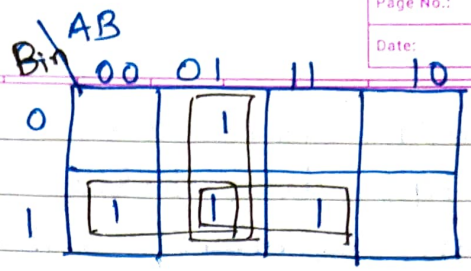
Truth-Table :-

Inputs			Outputs	
A	B	Bin	D	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

TG:SPPU_FE_ENGG



(a) K-Map for D



(b) K-map for Bout

- Therefore, the boolean Expression for D is as B.

$$D = \bar{A}B\bar{B}in + A\bar{B}\bar{B}in + \bar{A}\bar{B}Bin + ABBin$$
$$= \bar{B}in(\bar{A}B + A\bar{B}) + Bin(\bar{A}\bar{B} + AB)$$
$$= \bar{B}in(A \oplus B) + Bin(\overline{A \oplus B})$$

$\because A \oplus B = X$

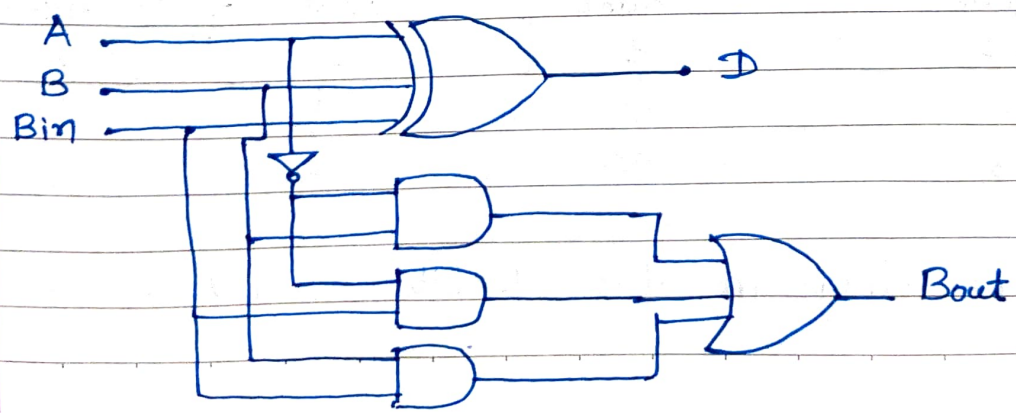
$$= \bar{B}in(X) + Bin(\bar{X})$$
$$= \bar{B}inX + Bin\bar{X}$$
$$= Bin \oplus X$$
$$= Bin \oplus A \oplus B$$

D $\Rightarrow A \oplus B \oplus Bin$ which becomes exoring of the three inputs after simplification.

- The boolean Expression for Bout is,

$$Bout = \bar{A}B + \bar{A}Bin + BBin$$

- There are two boolean Expressions can be used to design a combinational logic ckt for full-subtractor as shown in Fig.



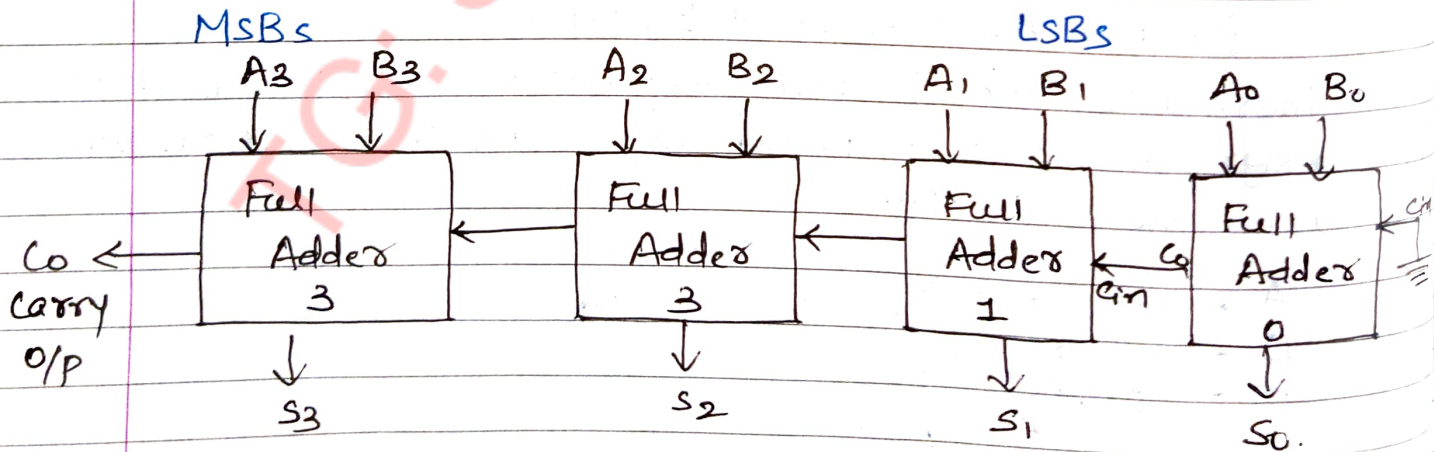
TG: SPUTTERING

⇒ Binary Adder :-

IC 7483 is a 4-bit parallel adder which consists of four interconnected full adders along with the look ahead carry ckt. The pin diagram of IC 7483 is shown in fig. It is a 16 pin IC. The inputs to the IC are A, B and Cin. while outputs are sum and Cout



→ The block diagram of a four bit parallel adder using full adders is shown in Fig.



Block diagram of a 4-bit binary adder

→ Let the two four bit words that are to be added be A and B and be donated as follows:

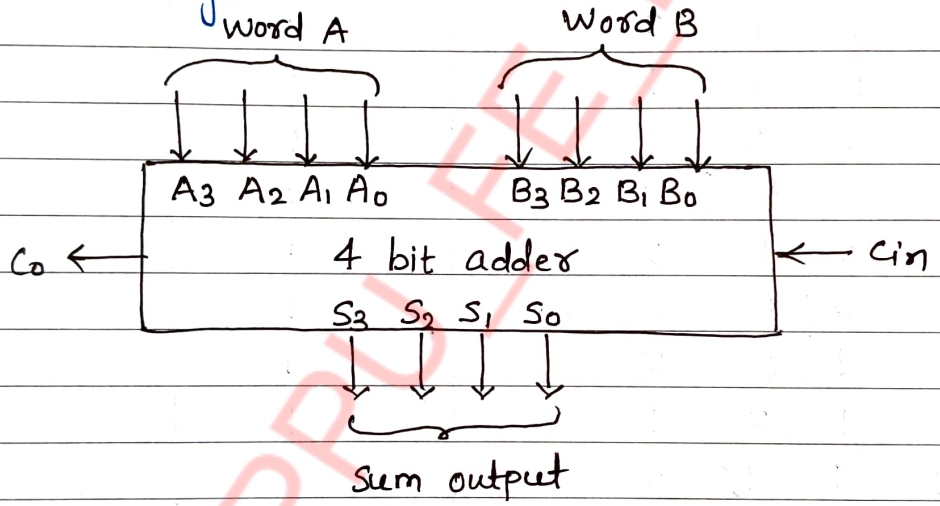
$A = A_3\ A_2\ A_1\ A_0$

$B = B_3\ B_2\ B_1\ B_0$

→ A_0 and B_0 represent the LSBs of the four bits words A and B. Hence full adder-0 is the lowest stage. Hence its Cin has been connected to '0' permanently.

→ The rest of connections are exactly same as those done for the 4-bit binary adder.

→ Block diagram:



→ It has two 4 bit inputs $A_3, A_2, A_1 .. A_0$ and $B_3, B_2, B_1, .. B_0$, a carry output and 4-bit sum o/p S_3, S_2, S_1, S_0.

⟹ BCD Adder :-

To implement BCD adder we require:

1. 4 bit binary adder for initial condition.
2. logic ckt to detect sum greater than 9 and one more 4-bit adder to add $(0110)_2$ in the sum if sum is greater than 9 or carry is 1.

- BCD adders can be cascaded to add numbers several digits long by connecting the carry out

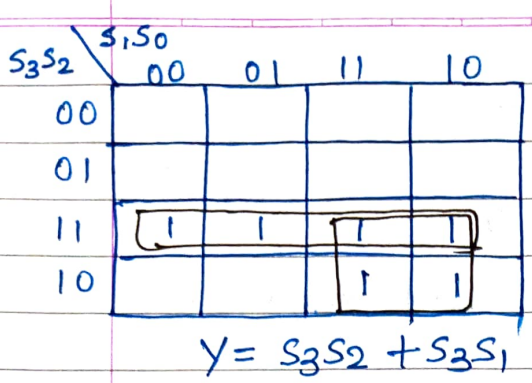
TG: SPPU FE ENGG

of a stage to the carry in of the next stage.

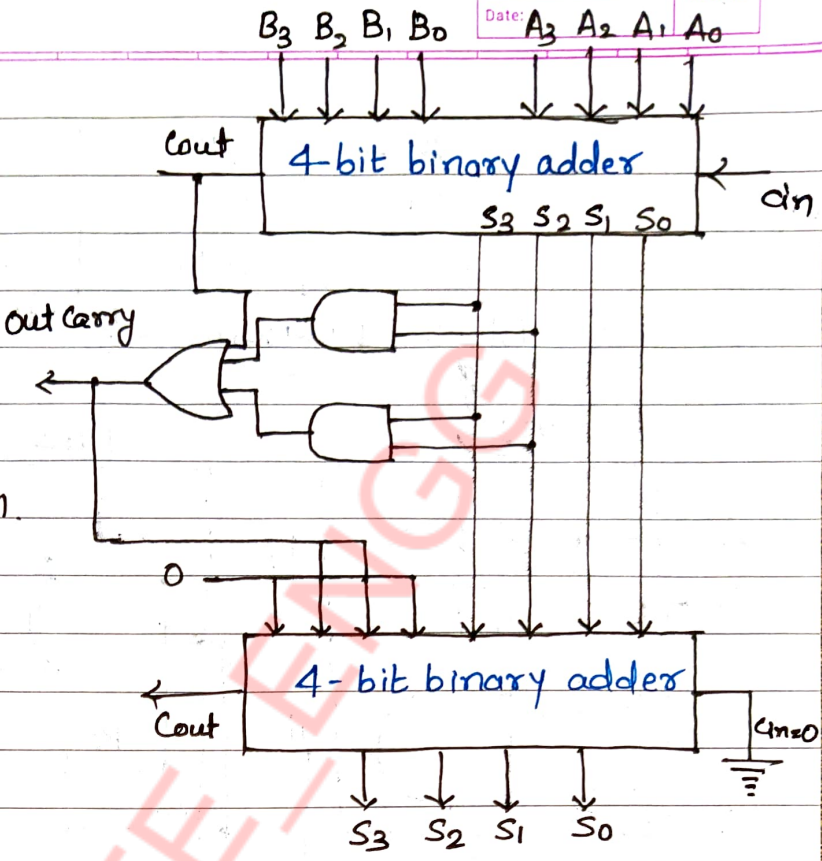
Input				Output
S_3	S_2	S_1	S_0	Y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

⇒ The logic ckt to detect sum greater than 9 can be determined by simplifying the boolean expression of given truth table. Y=1 indicates sum is greater than 9, we can put one more term Cout in the above expression. to check wheather carry is one. It any one condition is satisfied we need to add (0110) in the sum.

TG: SPPU_FE_ENGG



(a) K-Map Simplification.



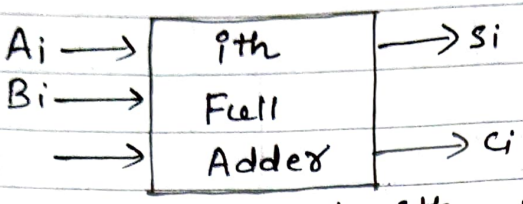
(b) Block diagram of BCD adder.

⇒ Carry Look - Ahead Adder :~

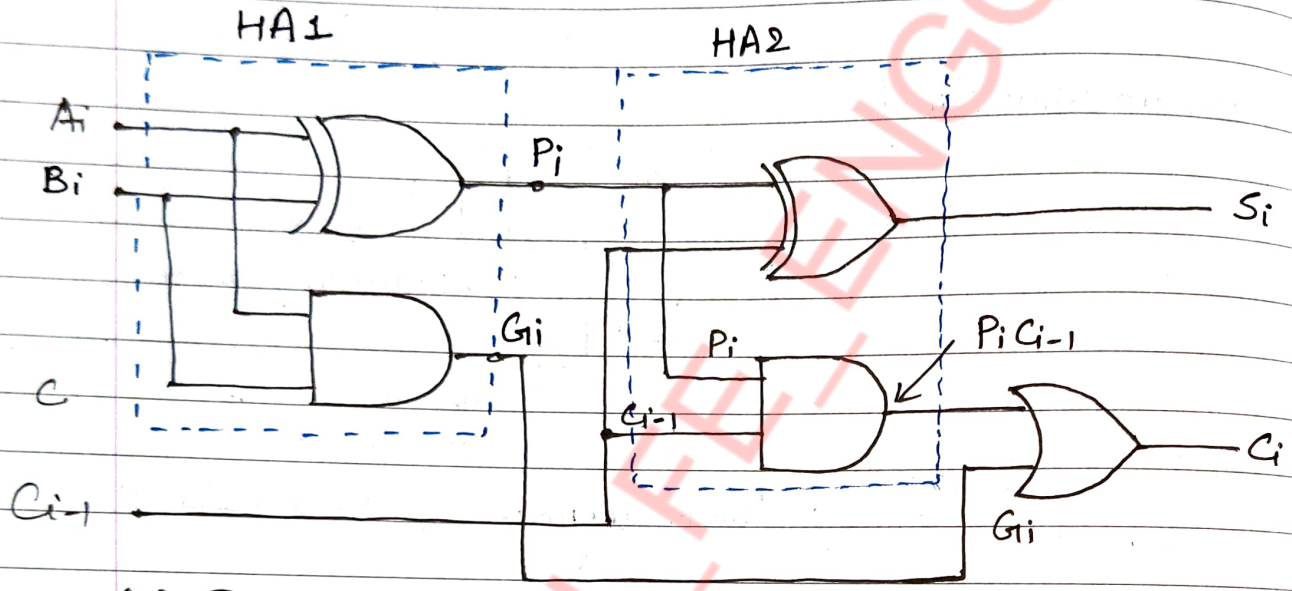
→ The delay generated by N-bit adder is proportional to the length N of the two Numbers X and Y that are added because the carry signals have to propagate from one full-adder to the next. For large values of N, the delay becomes unacceptly large so, that a special solution needs to be adopted to accelerate the calculation of the carry bits.

→ This solution involves a "look-ahead carry generator" which is a block that simultaneously calculates all the carry bits involved.

KG SPUTTERING



(a) Block Diagram of ith Full Adder



(b) Realization using AND-OR-EXOR gates.

→ Consider the block diagram of Full Adder fig(a) and its AND-OR-EXOR realization shown in Fig(b)

→ Refer fig. to write,

$$P_i = A_i \oplus B_i$$

and $G_i = A_i B_i$

Also $S_i = P_i \oplus C_{i-1} = A_i \oplus B_i \oplus C_{i-1}$

and $C_i = G_i + P_i C_{i-1}$

→ The carry output G_i of the first half adder is Equal to 1 if $A_i = B_i = 1$ and a carry is generated at the ith stage of the binary adder. That means $C_i = 1$.

TGI SPPU FE ENGG

M T W T F S S
Page No.:
Date:
YOUVA

→ This Variable G_i is known as Carry Generate and its Value does not depend on the input carry i.e. C_{i-1}.

→ The Variable P_i is called as Carry Propagate because this term is associated with the propagation of carry from C_{i-1} to C_i. Now consider Equation. i.e. $C_i = G_i + P_i C_{i-1}$.

$$P_i = A_i \oplus B_i$$

→ Stage expression for carry output.

0 $C_0 = G_0 + P_0 C_{-1}$

1 $C_1 = G_1 + P_1 C_0$

$= G_1 + P_1 (G_0 + P_0 C_{-1})$

$\therefore C_1 = G_1 + P_1 G_0 + P_0 P_1 C_{-1}$

→ Stage expression for carry output.

2 $C_2 = G_2 + P_2 C_1$

$= G_2 + P_2 (G_1 + P_1 G_0 + P_0 P_1 C_{-1})$

$\therefore C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_0 P_1 P_2 C_{-1}$

3 $C_3 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_{-1}$

Draw combine diag. which explained in the class

→ In the expressions stated above, the Variable involved are, $G_0, G_1, G_2, G_3, P_0, P_1, P_2, P_3$ and C_{-1}.

→ Out of them the G Variables are generated from the A and B inputs using AND gates

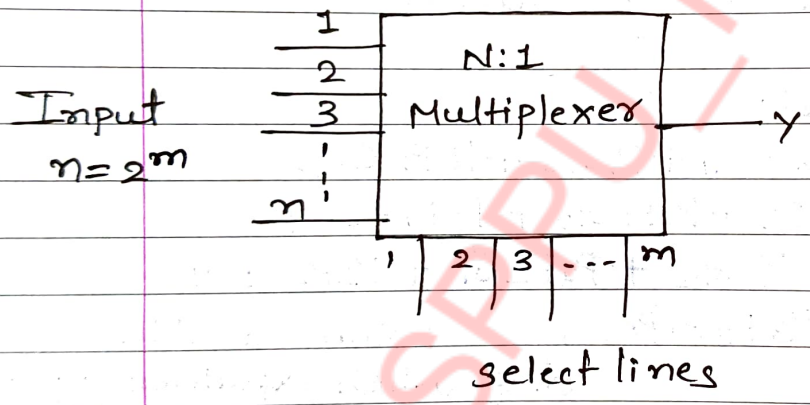
→ Aand the P Variables are obtained again directly from A and B inputs using EX-OR gates.

→ If the G, P and C_{-1} are at a time available, then it is possible to produce the carry outputs C_0, C_1, C_2 and C_3 by using 2-level realization (AND-OR or NAND-NAND) etc.

TO SPPU / FE ENGG

⇒ Multiplexers :-

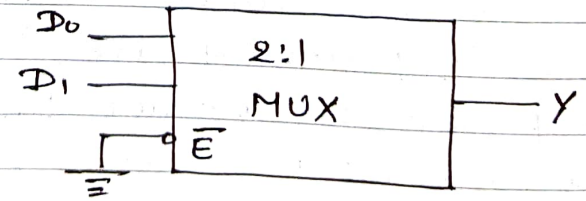
* The literal meaning of word 'multiplex' is 'Many into one'. A Multiplexer ckt may have several inputs and only one output.
* Multiplexer is a special combinational logic ckt. which accepts many inputs and allows only one of them to get through to the output at any instance of time.
* Therefore, multiplexer output is a particular data input which is selected with the help of control signal called 'select'.
* The symbolic representation of multiplexer is shown in Fig.



⇒ 2:1 Multiplexer

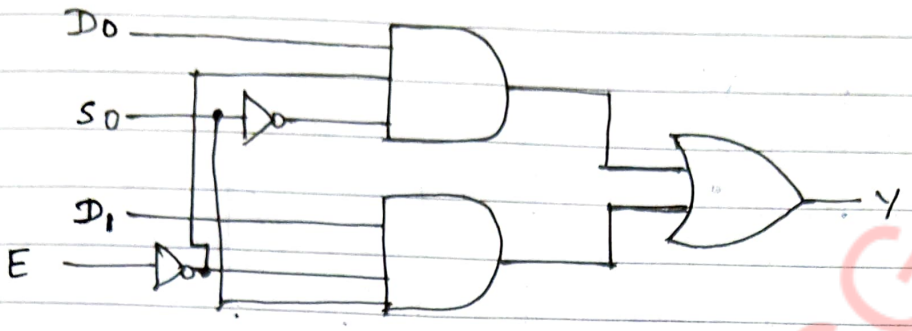
- There are two data inputs D_0 and D_1 in 2:1 multiplexer.
- It has only one output. The number of select lines (m) is equal to 1.
- Truth Table

S_0	Y
0	D_0
1	D_1



Logic Symbol of 2:1 MUX.

M T W T F S S
Page No.:
Date:
YOUVA



$$Y = \bar{S}_0 D_0 + S_0 D_1$$

logic ckt of 2:1 Multiplexer

⇒ 4:1 Multiplexer :~

- 4:1 Multiplexer ckt has four data inputs and one output
- The number of select lines required to control four data inputs is two.
- These two select lines are called S_0 and S_1.
- The Boolean expression for Y output is,

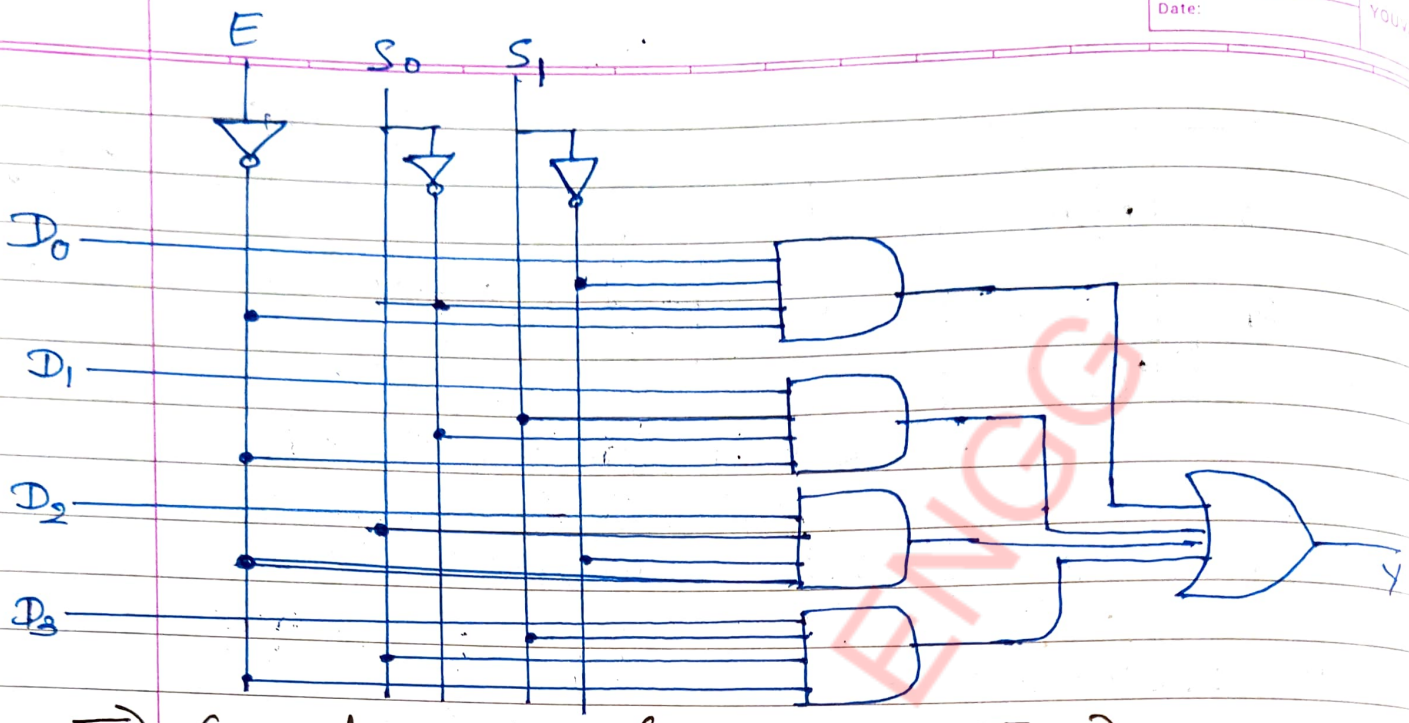
$$Y_1 = \bar{S}_1 \bar{S}_0 D_0 + \bar{S}_1 S_0 D_1 + S_1 \bar{S}_0 D_2 + S_1 S_0 D_3$$

→ Truth- Table :-

Inputs		Output
S_1	S_0	Y
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3



TG:SPPUEE_ENGG



⇒ Cascading MUX (Multiplexer Tree):-

Fig shows design of 8:1 Multiplexer using two 4:1 Multiplexers and one 2:1 Multiplexer.

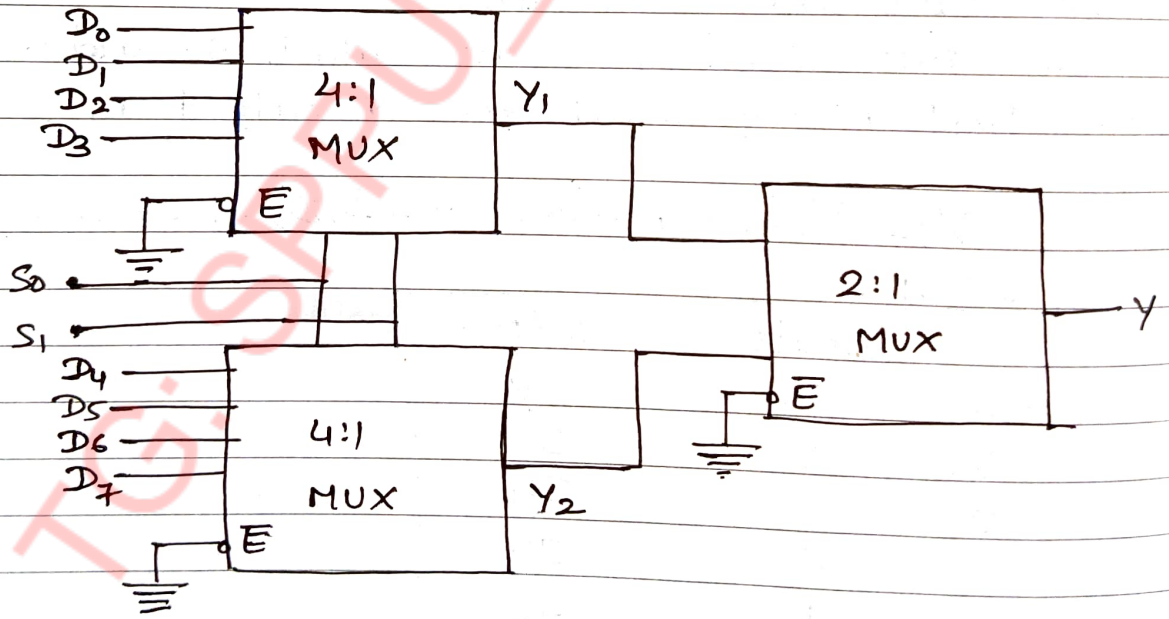


Fig. Multiplexer

TG:SPPU_FE_ENGG

M T W T F S S
Page No:
Date:
YOUVA

Ex:- Implement the following expression using a multiplexer.

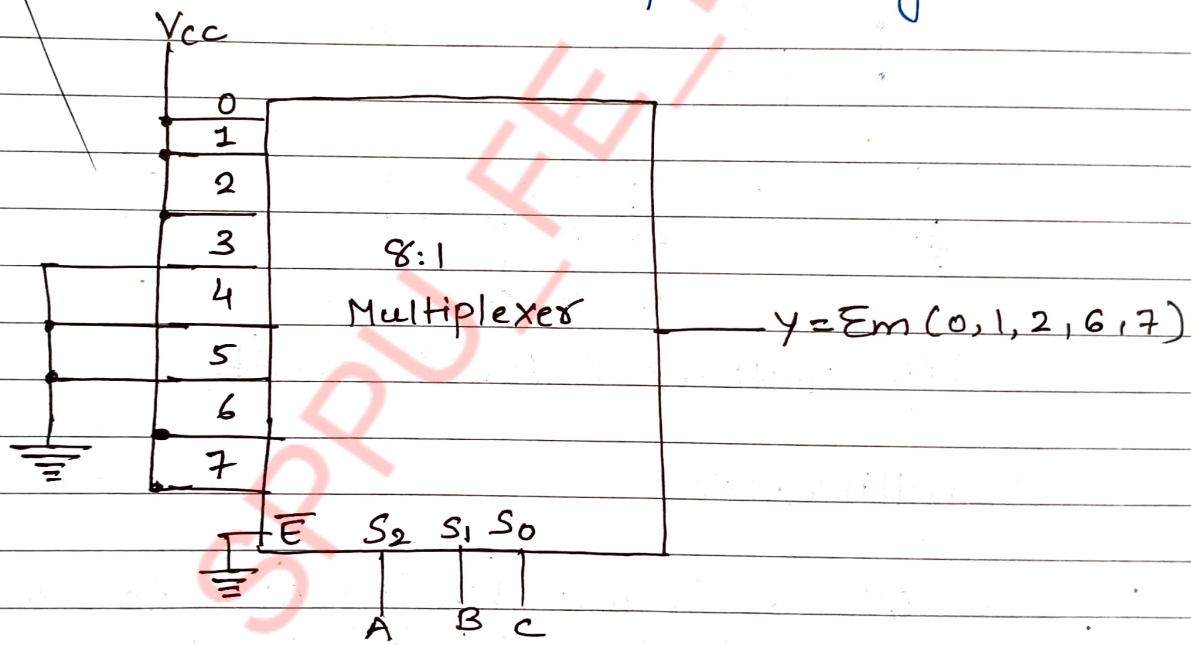
$$Y = \Sigma m\ (0, 1, 2, 6, 7)$$

Soln:-

Boolean Expression is,

$$Y(A, B, C) = \Sigma m\ (0, 1, 2, 6, 7)$$

It is a 3-Variable Boolean function and hence a multiplexer will require three select inputs. The inputs of Multiplexer corresponding to given minterms are connected to Vcc. other inputs are grounded.



⇒ Application of Multiplexer:-

1) They are used in time multiplexing system.
2) They are used in frequency Multiplexing systems.
3) It is also used implement combinational logic ckt.
4) They are used in data acquisition systems.

TG: SPPU_FE_ENGG

Ex:- Implement the following expression using a Multiplexer.

$$Y(A,B,C) = \pi M(0,1,4,5)$$

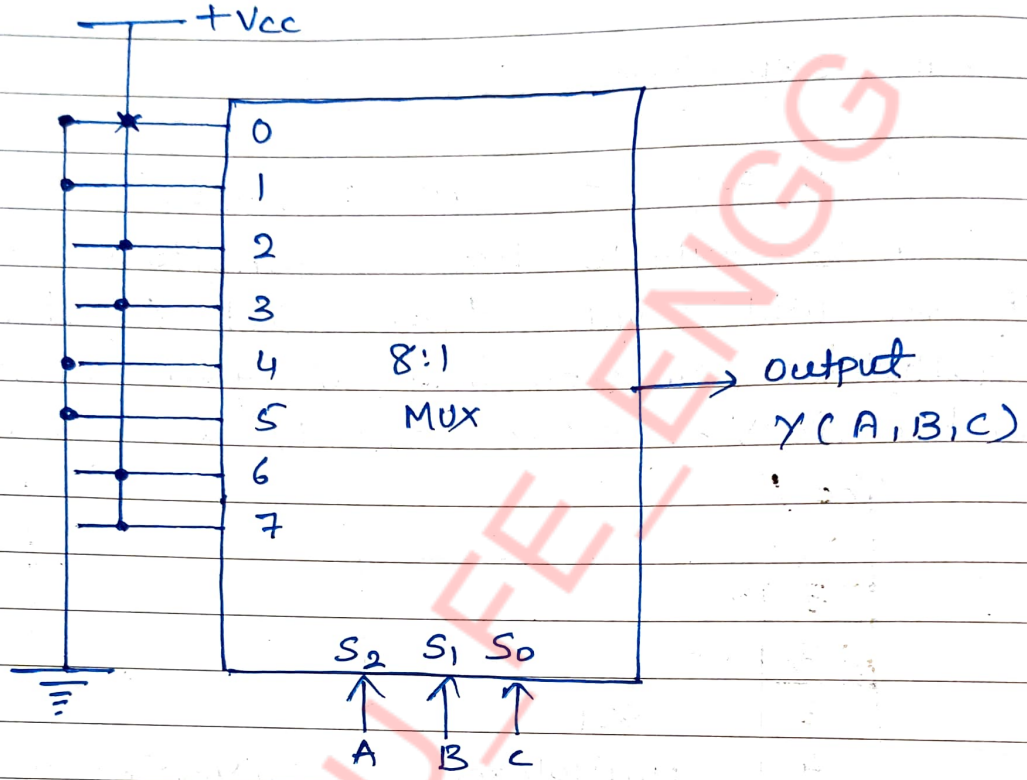


Fig. Multiplexer Based circuit.

⇒ Demultiplexers:-

- Demultiplexer is a combinational logic circuit having one input and several outputs.
- Demultiplexer Means 'one into many'
- It accepts a single input and sends it to one of the output lines.
- The output line is selected by control or select Signals. For n-output demultiplexer, number of select lines is m, where $n = 2^m$.
- Fig. depicts 1:n demultiplexer.

TG: SPPU FE_ENGG

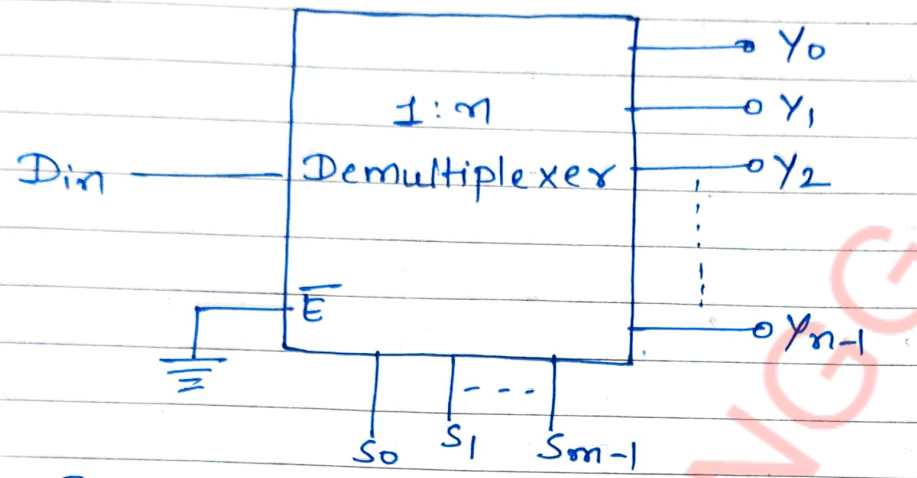


Fig. Block Diagram of 1:n Demultiplexer

→ Consider an example 1:4 demultiplexer.
→ The no. of select input lines is 2.
→ The truth-table of 1:4 demultiplexer is given Below:-

1:4 Demultiplexer

Select Inputs		Outputs			
S_0	S_1	Y_0	Y_1	Y_2	Y_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

→ The logic expression for different outputs can be written as,

$Y_0 = \overline{S_0}\,\overline{S_1}$, $Y_1 = \overline{S_0} S_1$, $Y_2 = S_0 \overline{S_1}$, $Y_3 = S_0 S_1$

→ The logic ckt diagram of 1:4 demultiplexer is shown in Fig.

TG: SPPU FE ENGG

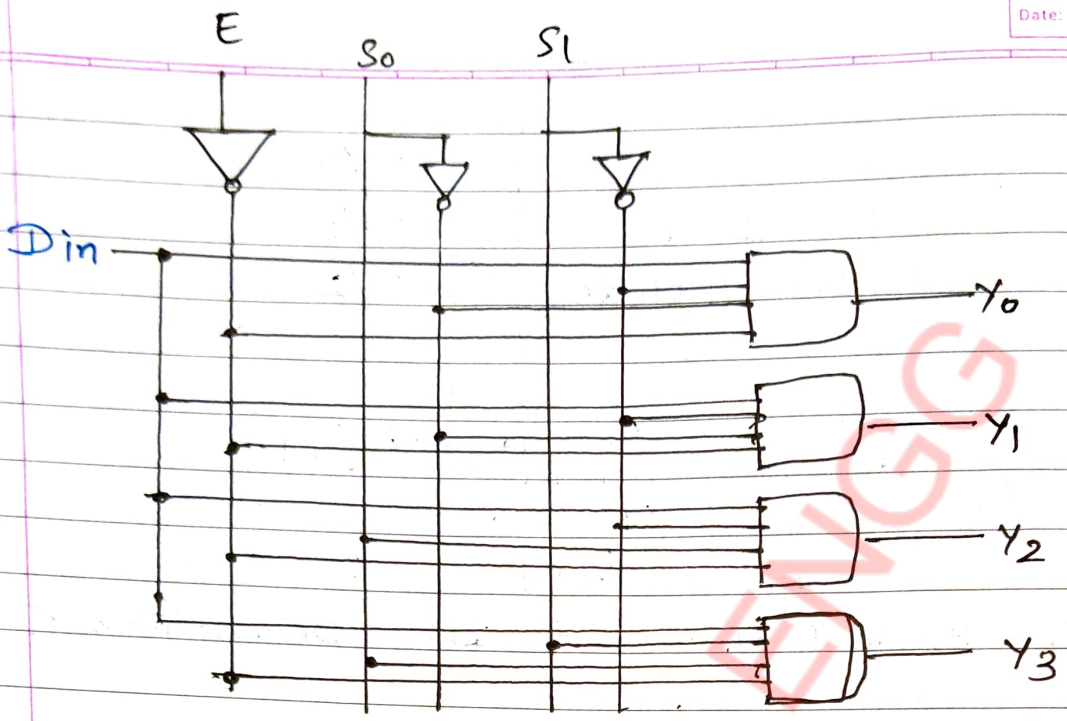


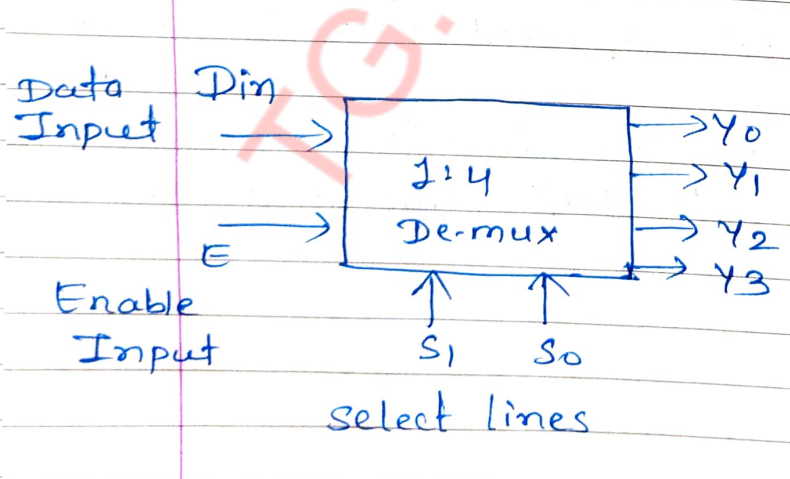
Fig:- 1:4 Demultiplexer

⇒ De-multiplexer as Decoder :~

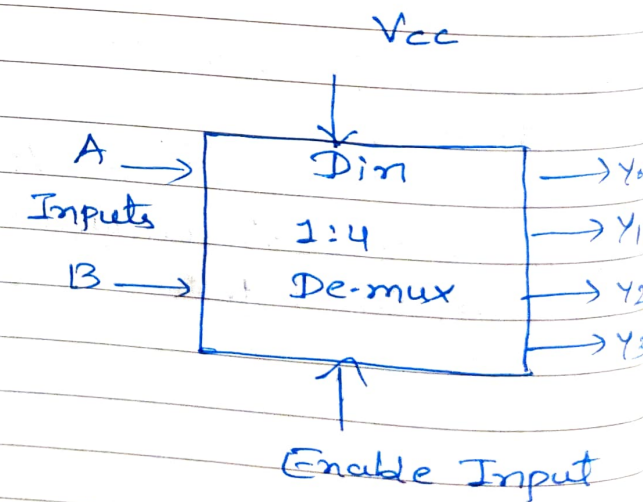
✓ It is possible to operate a de-multiplexer as a decoder.

✓ Let us consider an example of 1:4 demux can be used as 2:4 decoder.

⇒ 1:4 De-multiplexer as 2:4 Decoder :~



1:4 De-multiplexer



1:4 De-multiplexer as 2:4 Decoder

TG: SPPU FE ENGG

⇒ Realization of Boolean expression using De-mux.

→ We can implement any Boolean expression using de-multiplexers.

→ It reduces circuit complexity.

→ It does not require any simplification.

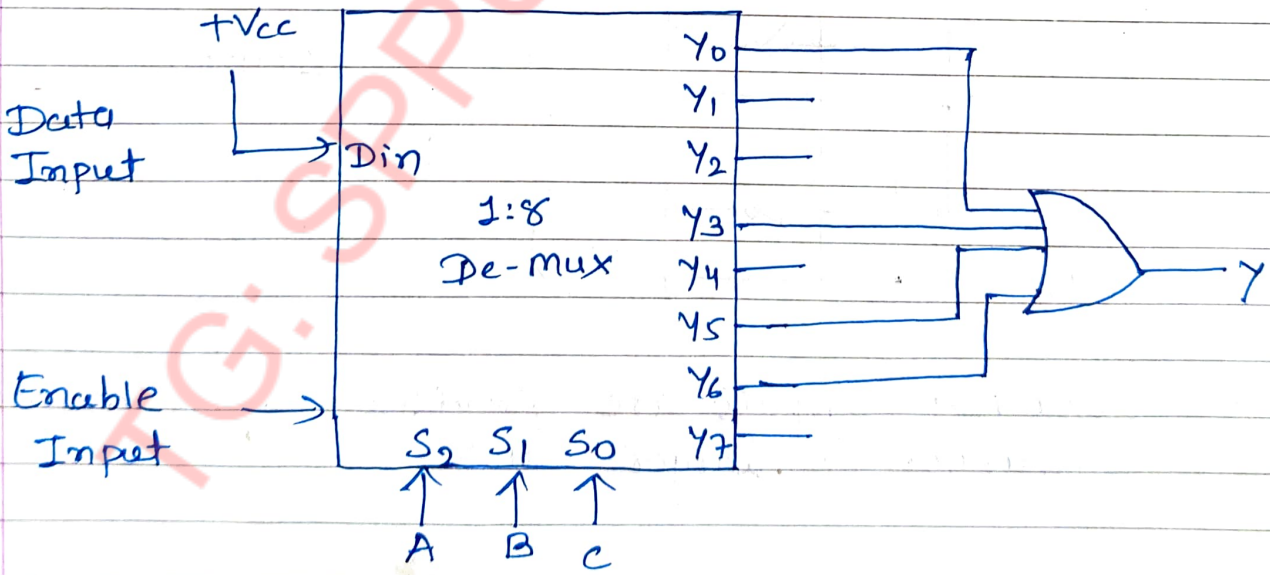
→ Example 1 :-

Implement following Boolean expression using de-multiplexer.

$$f(A,B,C) = \Sigma m(0,3,5,6)$$

✓ Since there are 3 variables, therefore a demultiplexer with 3 select input is required i.e. 1:8 de-multiplexer is required.

✓ The 1:8 de-multiplexer is configured as below to implement given Boolean expression.

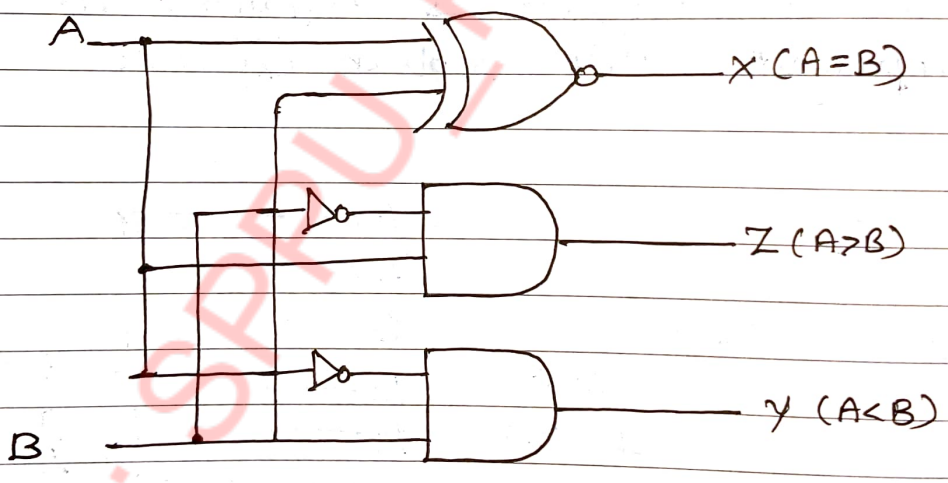


TG: SPPU_FE_ENGG

⇒ Comparators :-

- Comparators provide outputs to indicate which of the numbers is larger or if they are Equal.
- The bits are numbered starting at 0, rather than 1 as in the case of adders.

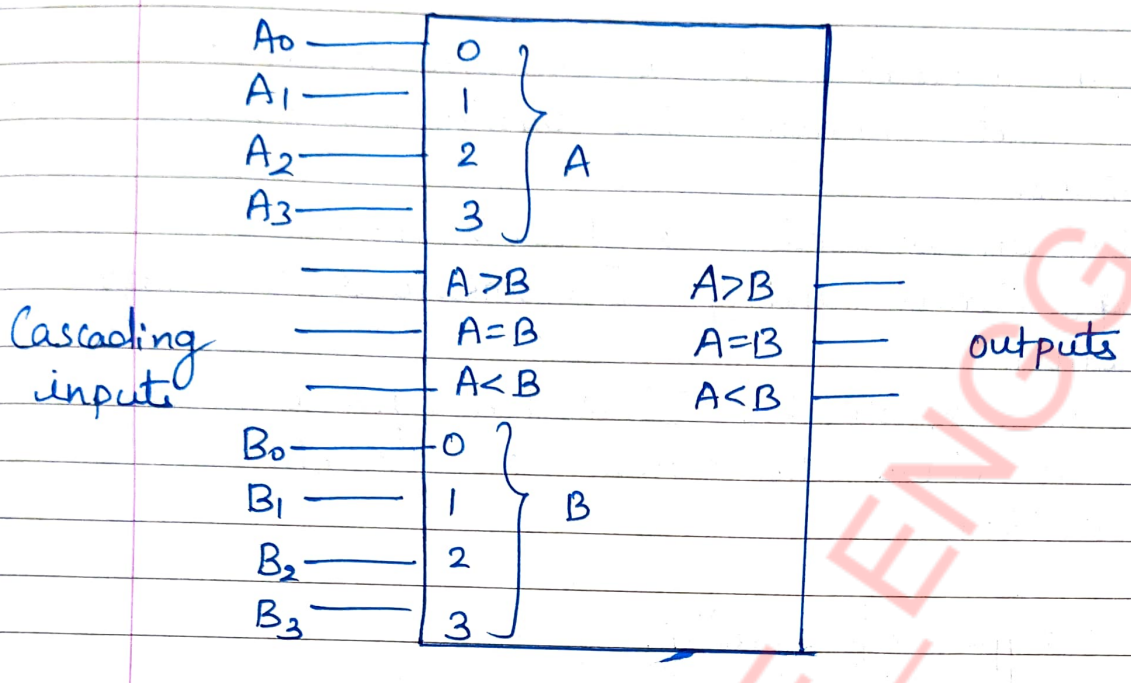
A	B	X A=B	Y A<B	Z A>B
0	0	1	0	0
0	1	0	1	0
1	0	0	0	1
1	1	1	0	0



- Cascading inputs are provided to expand the comparator to larger numbers.

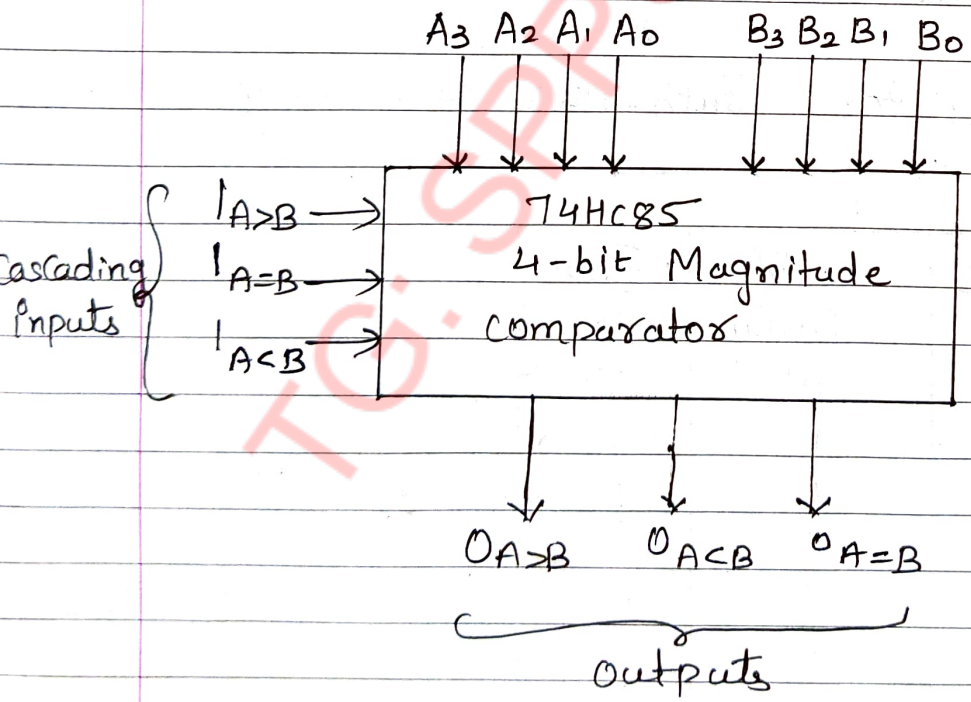
TG: SPRUFE_ENGG

M T W T F S S
Page No.:
Date:
YOUVA



⇒ 74LS85 (4-bit Magnitude Comparator)

The 74LS85 compares two unsigned 4-bit binary numbers, the unsigned numbers are A_3, A_2, A_1, A_0 and B_3, B_2, B_1, B_0



TG: SPPU_ENGG

⇒ What is parity Generator?

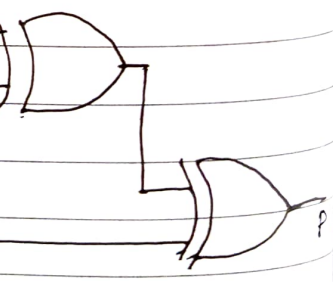
- A parity generator is a combinational logic circuit that generates the parity bit in the Transmitter.
- A parity bit is used for the purpose of Detecting Errors during Transmissions of binary Information.
- It is an Extra bit Included with a binary Message to Make the Number of 1's either Odd or Even.

⇒ Two types of parity

- In Even parity, the added parity bit will Make the Total Number of 1's an Even Amount.
- In Odd parity, the added parity bit will Make the total Number of 1's an odd Amount.

⇒ Parity Generator Truth-table and logic diagram.

3-bit Message X	Y	Z	Odd Parity Bit	Even Parity Bit
0	0	0	1	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	0	1



TG SPDU EE ENGG

Boolean Expression

K-Map Simplification.

⇒ Even pair

x \ yz	00	01	11	10
0	0	1	0	1
1	1	0	1	0

$P = \bar{x}\bar{y}z + \bar{x}y\bar{z} + x\bar{y}\bar{z} + xyz$

$= \bar{x}(\bar{y}z + y\bar{z}) + x(\bar{y}\bar{z} + yz)$

$= \bar{x}(y \oplus z) + x\,\overline{(y \oplus z)}$

$\therefore y \oplus z = A$

$= \bar{x}A + x\bar{A}$

$= x \oplus A$

$P = x \oplus y \oplus z$

⇒ Odd pair

x \ yz	00	01	11	10
0	1	0	1	0
1	0	1	0	1

$P = \bar{x}\bar{y}\bar{z} + \bar{x}yz + x\bar{y}z + xy\bar{z}$

$= \bar{x}(\bar{y}\bar{z} + yz) + x(\bar{y}z + y\bar{z})$

$= \bar{x}\,\overline{(y \oplus z)} + x(y \oplus z)$ $\quad y \oplus z = A$

$= \bar{x}\bar{A} + xA$

$= \overline{x \oplus A}$

$P = \overline{x \oplus y \oplus z}$

⇒ Parity checker :-

* A circuit that checks the parity in the Receiver is called parity checker.

* The parity checker circuit checks for possible Errors in the Transmission.

* Since, the Information Transmitted with Even parity, the Received must have an even number of 1's.

TG: SPPU_ENGG

If it has odd number of 1's, it indicates that there is a Error occured during Transmission.

- The output of the parity checker is denoted by parity Error checker. If there is error, that is, if it has odd number of 1's, it will indicate, 1. If no then PEC will Indicate 0.

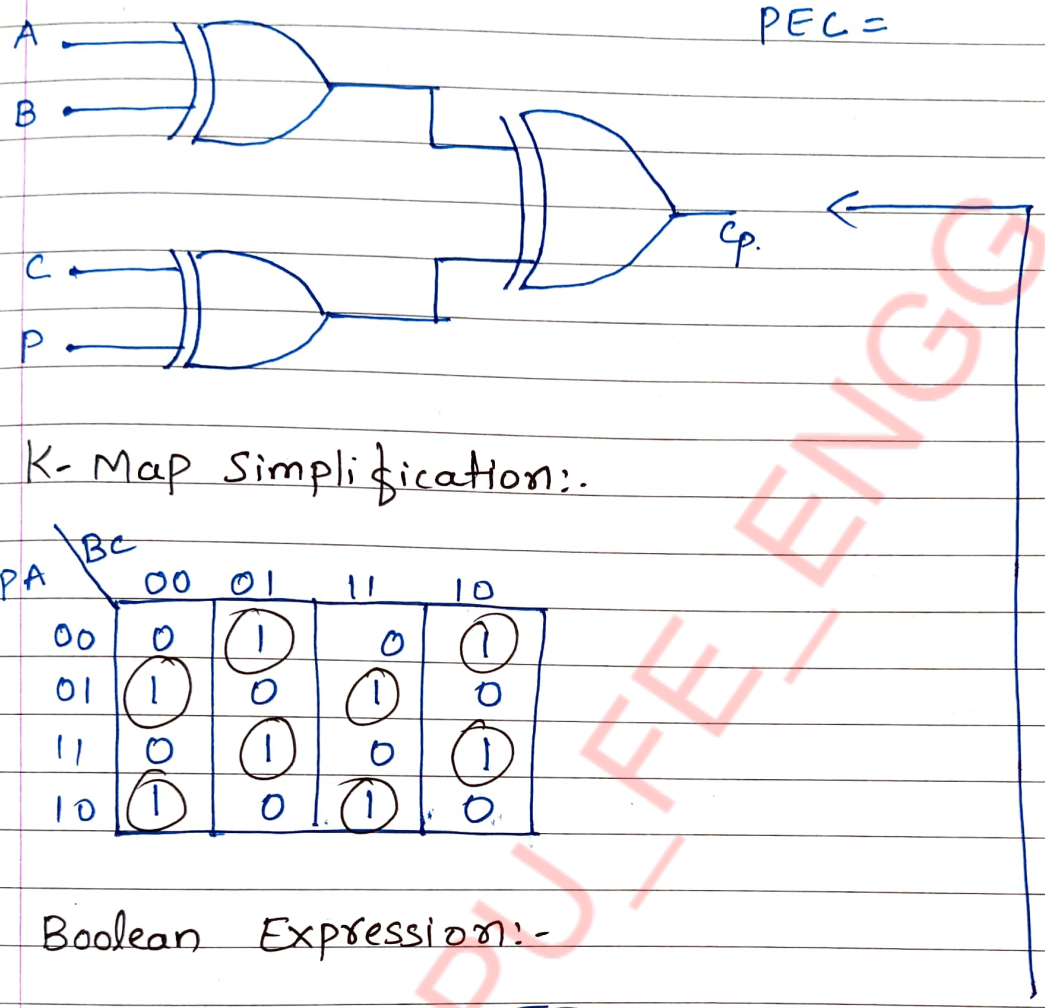
⇒ Even parity Checker Truth-Table:-

Decimal Equivalent	Four Bits Received				PEC
	P	A	B	C	
0	0	0	0	0	0
1	0	0	0	1	1
2	0	0	1	0	1
3	0	0	1	1	0
4	0	1	0	0	1
5	0	1	0	1	0
6	0	1	1	0	0
7	0	1	1	1	1
8	1	0	0	0	1
9	1	0	0	1	0
10	1	0	1	0	0
11	1	0	1	1	1
12	1	1	0	0	0
13	1	1	0	1	1
14	1	1	1	0	1
15	1	1	1	1	0

TG: SPPU_FE_ENGG

→ Logic Diagram

Boolean Expression



PEC =

→ K-Map Simplification:.

PA \ BC	00	01	11	10
00	0	1	0	1
01	1	0	1	0
11	0	1	0	1
10	1	0	1	0

→ Boolean Expression:-

$$PEC = \bar{P}\bar{A}\bar{B}C + \bar{P}\bar{A}B\bar{C} + \bar{P}A\bar{B}\bar{C} + \bar{P}ABC + PA\bar{B}C + PAB\bar{C} + P\bar{A}\bar{B}\bar{C} + P\bar{A}BC$$

$$= \bar{P}\bar{A}(\bar{B}C + B\bar{C}) + \bar{P}A(\bar{B}\bar{C} + BC) + PA(\bar{B}C + B\bar{C}) + P\bar{A}(\bar{B}\bar{C} + BC)$$

$$= \bar{P}\bar{A}(B \oplus C) + \bar{P}A(\overline{B \oplus C}) + PA(B \oplus C) + P\bar{A}(\overline{B \oplus C})$$

$$= (B \oplus C)(\bar{P}\bar{A} + PA) + (\overline{B \oplus C})(\bar{P}A + P\bar{A})$$

$$= (B \oplus C)(\overline{P \oplus A}) + (\overline{B \oplus C})(P \oplus A)$$

$B \oplus C = X$

$P \oplus A = Y$

$$PEC = X\bar{Y} + \bar{X}Y = X \oplus Y = (B \oplus C) \oplus (P \oplus A)$$

TG:SPPU_FE_ENGG

Given 20/08/24 Submission 27/08/24

Assignment No 2

1. Draw and Explain the look ahead carry generator.
2. Design and implement Binary to gray code converter using logic gates.
3. Implement the expression using a 8:1 Multiplexer
   $F(a, b, c, d) = \Sigma m\ (0, 2, 3, 6, 8, 9, 12, 14)$
4. Design using Single 4:1 multiplexer and logic gates: (Scan book Pg. No. 2.15)
5. How will you implement full-adder using half-adder? Explain the circuit diagram.
6. Design full adder using 8:1 multiplexer and draw the diagram.
7. Design two bit comparator using gates. (Consider A_1 MSB and A_0 LSB).

TG: SPPU_FE_ENGG